

QA AUTOMATION FRAMEWORK

Getting Started Guide

How AI-assisted test automation with Playwright and Claude works, and how to set up and use this framework from scratch.

Audience	QA engineers and developers joining the project; team leads who want to know how it works
Applies to	Web applications, mobile web, native Android and iOS apps
Tooling	Playwright, Appium (WebdriverIO), Claude Code + Playwright MCP, Allure 3, TypeScript, Node.js
Companion	<i>Module Testing & Automation Guide</i> (QA-Automation-Guide.pdf): the process and what developers put in a module MD file
Version	1.0 · 5 October 2026

Contents

1	The big picture	Everyone
2	How Playwright + Claude AI automation works	Concepts
3	Setup from scratch	Once per machine
4	How the framework is built	Architecture
5	Walkthrough: automating a module with Claude	Day-to-day work
6	Working in sprints	Per sprint
7	Adding a new application	Per project
8	Writing and editing tests by hand	Reference
9	Native mobile tests (Appium)	Optional
10	Running tests locally and in CI	Reference
11	Reports and failure triage	QA & developers
12	Rules of the framework	Must read
13	Working well with Claude	Tips
14	Troubleshooting	FAQ
A	Command cheat sheet	Print me
B	Glossary	Reference

The one-sentence version

Requirements arrive as Markdown. Claude turns them into test cases for QA to review, then into Playwright tests by opening the real application in a browser to find the elements. The tests run on web, mobile web and native apps, and results land in an Allure report linked to Jira.

Fastest path for a new team member

Read sections 1 and 2 (15 minutes) to understand the idea.

Follow section 3 to install everything and run the sample app's tests.

Read section 4 with the sample app `apps/saucedemo` open next to it.

Do the walkthrough in section 5 on a small module of your own app.

1. The big picture

1.1 The problem this solves

Hand-written UI automation is slow to build and breaks easily. Most of the effort goes into repetitive work: reading the story, listing test cases, finding locators for every field, writing page objects and keeping them up to date when the screen changes. This framework hands that repetitive work to an AI coding assistant (Claude), inside a fixed structure, so that:

- every module follows the same path: **requirement** → **test cases** → **automated tests** → **report**;
- QA engineers review and decide; Claude drafts, writes code, runs tests and fixes automation problems;
- one shared framework serves every application in the company, web and mobile;
- each test is traceable: Jira story → test-case ID → test → result in the report.

1.2 What is in the box

Piece	What it does
Playwright	Drives real browsers (Chrome, Safari engine) and runs the tests. Also emulates phones for mobile web.
Appium (via WebdriverIO)	Drives native Android and iOS apps on emulators or devices.
Claude Code	The AI assistant in VS Code. It reads the repository, writes code, runs commands and follows the project rules in <code>CLAUDE.md</code> .
Playwright MCP	A plug-in that lets Claude open and operate a real browser while it works, so it sees the actual page instead of guessing.
QA slash commands	<code>/qa-testcases</code> , <code>/qa-automate</code> , <code>/qa-fix</code> , <code>/qa-update</code> : saved, repeatable instructions for each step of the process.
Shared framework (<code>src/</code>)	Generic building blocks: field models, page/screen base classes, actions, assertions, login handling, API client, report enrichment.
Allure 3	The test report: grouped by Epic → Feature → Story, with Jira links, readable steps and a bug report for every failure.
Helper scripts	New app scaffolding, test-case coverage, and requirement change tracking (what changed since QA last tested).

1.3 Who does what

Role	Responsibilities
Developer	Writes the module MD file (fields, rules, exact messages, test IDs) and adds test IDs in the code. Details: companion guide. In sprint projects where only QA uses the framework, developers send their own notes with each build and QA merges them (chapter 6).
QA engineer	Runs the Claude commands, reviews test cases and generated code, triages failures, raises bugs, signs off modules.
Claude	Drafts test cases, explores the app in a browser, writes models/pages/data/specs, runs and fixes tests. Everything it produces is reviewed by QA.

Role	Responsibilities
Framework owners	Maintain <code>src/</code> , the commands and the conventions for every app.

2. How Playwright + Claude AI automation works

This section explains the general idea, independent of our framework. Once you understand these five building blocks, the rest of the guide is detail.

2.1 Playwright: the test engine

[Playwright](#) is an open-source library from Microsoft that controls a browser through code. A Playwright test is a TypeScript function that opens pages, interacts with them and checks the result:

```
test('valid user can log in', async ({ page }) => {
  await page.goto('https://www.saucedemo.com');
  await page.getByTestId('username').fill('standard_user');
  await page.getByTestId('password').fill('secret_sauce');
  await page.getByTestId('login-button').click();
  await expect(page).toHaveURL(/inventory/); // web-first assertion: retries until true or
  timeout
});
```

The key ideas:

- **Locators** describe *how to find* an element: by test ID, by accessible role and name, by label, by text. Stable locators are the single biggest factor in stable tests.
- **Auto-waiting**: actions wait until the element is visible, enabled and stable, so tests need no `sleep`.
- **Web-first assertions**: `expect(...)` retries until the condition is true or the timeout expires.
- **Projects**: the same tests run on several browsers and devices (desktop Chrome, Pixel 7, iPhone 15).
- **Fixtures**: reusable setup handed to each test (a page, a logged-in session, an API client).
- **Trace, screenshots and video** are recorded on failure, so you can replay exactly what happened.

2.2 Appium: the same idea for native apps

Browsers can't run native Android/iOS apps, so for those we use **Appium**, a server that controls apps on an emulator, simulator or real device. We talk to it with WebdriverIO, but the framework hides this: mobile tests are written exactly like web tests (models, screens, `act / verify`), and Playwright's runner still runs them and produces the same report.

2.3 Claude Code: the AI that writes and runs the tests

Claude Code is Anthropic's AI coding assistant. In VS Code it works in a side panel and, unlike a chat bot, it can **read and edit files in the project and run terminal commands** (with your permission). That makes it an agent: you give it a goal ("automate these test cases"), and it reads the inputs, writes the code, runs the tests, reads the failures and fixes them, in a loop, until the goal is reached.

Two project files steer its behaviour:

- **CLAUDE.md** (repository root): the project rules, read automatically at the start of every conversation. Folder layout, naming, "locators only in models", "no sleeps", how to mark a known bug. This is why Claude's code looks like the rest of the framework.
- **Slash commands** (`.claude/commands/*.md`): saved, step-by-step instructions you start by typing `/qa-automate <file>`. Each step of our process has one, so everybody gets the same result.

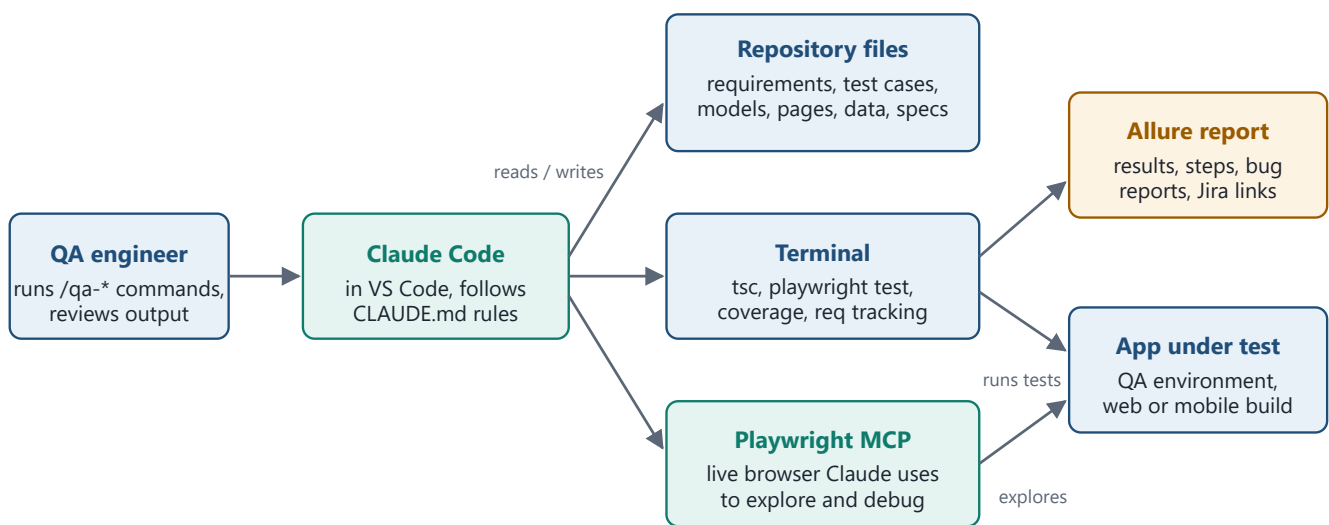
2.4 MCP and the Playwright MCP: letting Claude see the application

MCP (Model Context Protocol) is an open standard for giving an AI assistant extra tools. The **Playwright MCP** server gives Claude a real browser it can drive while it works: open a URL, read the page structure (an *accessibility snapshot*), click, type, run JavaScript in the page, take screenshots.

Why it matters: without it, Claude would have to *guess* locators and message texts from the requirement. With it, Claude logs in to the QA environment with a test account, opens the screen, lists the real test IDs, triggers each validation error to read its exact text, and only then writes the model. When a test fails, it can replay the steps in the browser to tell an automation problem from an application bug.

The server is configured for this project in `.mcp.json`, and `.claude/settings.json` pre-approves its tools. Native mobile apps can't be opened through it; for those, test IDs come from the module MD file or Appium Inspector.

2.5 How the pieces work together



Claude edits files, runs tests in the terminal and looks at the live app through the MCP browser. The tests themselves are plain Playwright code: they run without Claude, locally and in CI.

Important: the AI writes the tests, it doesn't run them in production

The generated tests are ordinary TypeScript files committed to the repository. Running them, locally or in CI, needs **no AI at all**. Claude is only involved when tests are created, changed or investigated.

2.6 Where AI helps, and where humans must decide

Claude is good at

- Turning a structured requirement into a complete list of positive, negative and boundary cases
- Finding stable locators on a real page
- Writing consistent model / page / data / spec code
- Running tests, reading errors, fixing locators and waits
- Spotting gaps in a requirement ("no error text given for...")

QA must decide

- Whether the test cases are right and complete (review step)
- What the **expected** result is: the requirement wins, never the current app behaviour
- Whether a failure is a real bug, and raising it in Jira
- Merging generated code (review it like any pull request)
- Signing off a module

Never let a test be "fixed" by weakening the check

If the application shows the wrong message, the test must keep expecting the right one. Confirmed bugs are marked with `knownBug('JIRA-KEY', 'summary')` (section 11.3), not by changing the assertion to match the bug. The commands are written to enforce this; reviewers should check it too.

3. Setup from scratch

Do this once per machine. It takes about 20 minutes, plus the optional mobile setup.

3.1 Prerequisites

Tool	Version / note	Check
Node.js	20 or newer (LTS recommended)	<code>node -v</code>
VS Code	Latest	
Claude Code extension	VS Code Marketplace: Claude Code (publisher: Anthropic). Needs a Claude account from the team's plan.	Claude icon in the side bar
Git	To get the code and commit work	<code>git --version</code>
Java	Not needed: Allure 3 runs on Node.js	
Android Studio / Xcode, Appium	Only for native mobile tests (section 9)	<code>appium -v</code>

3.2 Install the project

```
# 1. get the code (clone the repository, or unzip the shared copy) and open the folder in VS Code
cd "Automation Framework"

# 2. install dependencies and browsers
npm install
npm run setup                # downloads Chromium + WebKit for Playwright

# 3. create your local settings files from the examples (never commit the real ones)
copy .env.example .env      # macOS/Linux: cp
copy apps\saucedemo\.env.example apps\saucedemo\.env
```

The root `.env` chooses the app and environment:

```
APP=saucedemo                # folder name under apps/
TEST_ENV=qa                  # loads apps/<app>/.env.qa (if it exists) before apps/<app>/.env
JIRA_BASE_URL=https://yourcompany.atlassian.net # turns Jira keys in the report into links
```

Each app's own `apps/<app>/.env` holds its QA URLs, test accounts per role, API token and mobile app builds. Ask the QA lead for the values of real projects. They contain QA test accounts only, never production credentials.

3.3 First run

```
npx tsc --noEmit              # type check: should print nothing
npm run test:web               # sample app tests on desktop Chrome (logs in first, then runs the specs)
npm run report                 # builds and opens the Allure report in your browser
```

If you see green tests and a report grouped under *Authentication* and *Shopping*, the framework works.

3.4 Set up Claude Code and the browser tool

1. Install the **Claude Code** extension in VS Code and sign in with your Claude account.
2. Open the project folder (the one with `CLAUDE.md`) and open the Claude panel.
3. The project's `.mcp.json` declares the **playwright** MCP server. When Claude asks to enable it, approve it.
4. Type `/mcp` in the Claude panel: **playwright** should be listed as connected.
5. Type `/` and check that `qa-testcases`, `qa-automate`, `qa-fix` and `qa-update` appear.

Playwright MCP not listed or not connecting?

Add it for your user once, then reload VS Code and start a new Claude conversation:

```
claude mcp add playwright --scope user -- cmd /c npx -y @playwright/mcp@latest --isolated --  
output-dir=.playwright-mcp
```

On macOS/Linux leave out `cmd /c` (the same applies to `.mcp.json`, which is written for Windows). The first start downloads the package and can time out; retry once it has finished.

3.5 Try Claude on the sample app

Ask Claude, in plain words, to explain or run something. Good first prompts:

- "Explain how `apps/saucedemo/tests/web/login/login.spec.ts` finds its test case in the report."
- "Run the products tests on web-chrome and summarise the result."
- "Open `saucedemo.com` in the browser and list the data-test IDs on the login page." (uses the MCP browser)

4. How the framework is built

4.1 Folder layout

src/	SHARED FRAMEWORK: generic only, used by every app (never app-specific code)
config/	env.ts (only place reading process.env), app.ts (defineApp, settings),
paths.ts	
models/	field.types.ts: Field, WebLocator, MobileLocator, FormData, defineWebFields
web/	BasePage, WebActions (act), WebAssertions (verify), locator resolver
mobile/	BaseScreen, MobileActions, MobileAssertions, Appium driver + capabilities
api/	ApiClient (REST for setup / cleanup)
auth/	auth.setup.ts: logs in every role from app.config.ts
fixtures/	core `test`: role, platform, api, cleanup, driver, report (automatic)
report/	Allure enrichment: test case per test, failure cause, bug report attachment
utils/	step(), storyInfo() / knownBug(), cleanup, random data
apps/<app>/	ONE APPLICATION (copy of this structure per project)
app.config.ts	platforms, web base URL, test-id attribute, login roles + login steps
fixtures.ts	adds this app's pages/screens to the core test; specs import from here
.env	URLs, accounts, builds for this app (not committed)
requirements/	module MD files (+ .baseline/ = versions QA has processed)
test-cases/	reviewed test cases (*.testcases.md)
sprints/	sprint trackers + developer MDs as received (from-dev/<date>/)
models/web mobile	field models pages/ screens/ page and screen objects
data/	typed test data tests/web mobile specs
templates/	requirement.md, testcases.md, sprint.md, dev-handover.md
scripts/	new-app, tc-coverage, req-track
.claude/commands/	the /qa-* slash commands CLAUDE.md rules Claude follows
docs/	this guide and the module process guide

Imports use two aliases: `@core/...` for `src/` and `@apps/<app>/...` for app code.

4.2 How settings are resolved

APP picks the folder under `apps/`. Then settings are loaded, first value wins:

1. real environment variables (CI secrets, or set on the command line)
2. `apps/<app>/.env.<TEST_ENV>`, e.g. `.env.staging`
3. `apps/<app>/.env`
4. root `.env` (shared values such as `JIRA_BASE_URL`)

`BASE_URL` and `TEST_ID_ATTRIBUTE` in an env file override the defaults in `app.config.ts`, so one app can be tested against QA, staging, etc. In code, read settings only through `getEnv('NAME')` / `requireEnv('NAME')` from `@core/config/env`.

4.3 Platforms become Playwright projects

`platforms` in the app's `app.config.ts` decides which test projects exist:

Platform	Project(s)	Runs	Tests folder
web	web-chrome	Desktop Chrome	tests/web
mobile-web	mobile-web-android, mobile-web-ios	Responsive web on Pixel 7 (Chromium) and iPhone 15 (WebKit)	

Platform	Project(s)	Runs	Tests folder
android	android-app	Native app via Appium	tests/mobile
ios	ios-app	Native app via Appium (macOS only)	

Web projects depend on a `setup` project that logs in every role once and saves the sessions in `.auth/<app>/<role>.json`. Each test then starts already logged in, which keeps tests fast.

4.4 The four layers of every module

Each module is built from four small files. Examples are from the sample app `apps/saucedemo`.

1 Model: what is on the screen (the only place locators live)

```
// apps/saucedemo/models/web/login.model.ts
export const LoginFields = defineWebFields({
  username: { label: 'Username', type: 'text', locator: { testId: 'username' } },
    rules: { required: true, messages: { required: 'Epic sadface: Username is required' } }
},
  password: { label: 'Password', type: 'password', locator: { testId: 'password' } },
  loginButton: { label: 'Login button', type: 'button', locator: { testId: 'login-button' } },
  errorMessage: { label: 'Error message', type: 'label', locator: { testId: 'error' } },
});
export const LoginMessages = { invalidCredentials: 'Epic sadface: Username and password do not match ...' } as const;
export type LoginData = FormData<typeof LoginFields>; // { username?: string; password?: string }
(dereived)
```

The `label` is what appears in the report (Fill "Username" with "standard_user"). The `type` tells the framework how to fill the field. The data type is derived from the model, so data and model can't drift apart.

2 Page (web) or screen (mobile): business actions

```
// apps/saucedemo/pages/LoginPage.ts
export class LoginPage extends BasePage {
  readonly path = '/';
  readonly fields = LoginFields;

  async login(data: LoginData) {
    await this.act.fillForm(this.fields, data); // fills each field present in data, by its type
    await this.act.click(this.fields.loginButton);
  }
  async expectError(message: string) {
    await this.verify.text(this.fields.errorMessage, message);
  }
}
```

Pages use only `this.act.*` (actions) and `this.verify.*` (assertions). Both write a readable report step for every call. Each page/screen is registered once in `apps/<app>/fixtures.ts` so tests can ask for it by name.

3 Data: typed by the model

```
// apps/saucedemo/data/web/login.data.ts
export const validUser: LoginData = { username: getEnv('WEB_USERNAME'), password:
getEnv('WEB_PASSWORD') };
export const invalidLoginCases = [
```

```
{ id: 'TC-LOGIN-04', title: 'Empty username shows required error',
  data: { password: getEnv('WEB_PASSWORD') },           // username left out = left empty
  error: LoginFields.username.rules.messages.required },
// ...
};
```

4 Spec: reads like the test case

```
// apps/saucedemo/tests/web/login/login.spec.ts
import { test } from '@apps/saucedemo/fixtures';           // never from '@playwright/test'

test.use({ role: null });                                   // login tests start logged out

test.describe('Login', () => {
  test.beforeEach(async ({ loginPage }) => {
    await storyInfo({ epic: 'Authentication', feature: 'Login', story: 'SCRUM-101 User can log in',
      jira: 'SCRUM-101', severity: 'critical' });
    await loginPage.open();
  });

  test('TC-LOGIN-01 | Valid user is taken to the Products page', { tag: ['@smoke', '@TC-LOGIN-01'] },
    async ({ loginPage, inventoryPage }) => {
      await loginPage.login(validUser);
      await inventoryPage.expectLoaded();
    });

  for (const tc of invalidLoginCases) {                   // data-driven negative cases
    test(`${tc.id} | ${tc.title}`, { tag: ['@regression', `@${tc.id}`] }, async ({ loginPage }) => {
      await loginPage.login(tc.data);
      await loginPage.expectError(tc.error);
    });
  }
});
```

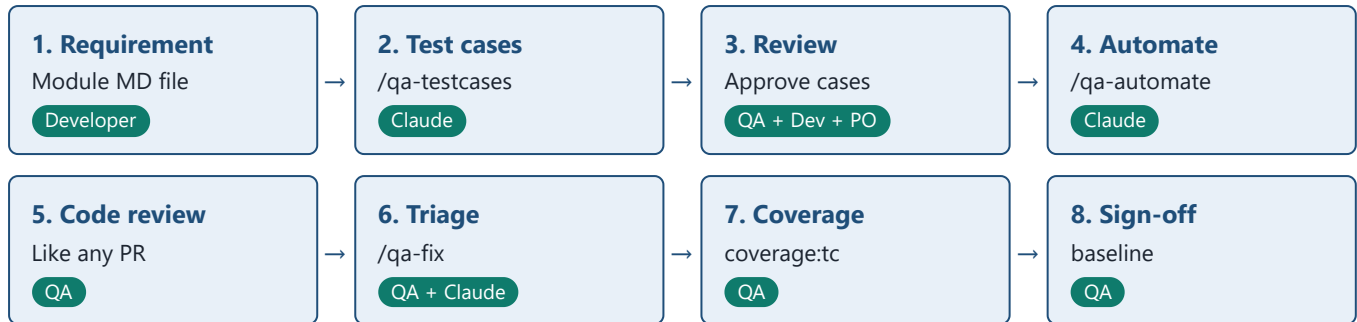
One test per test-case ID, titled `<ID> | <Title>`. The report uses the ID in the title to find the test case in the `*.testcases.md` file and show its steps and expected result next to the result.

4.5 What every test gets for free (fixtures)

Fixture / option	What it does
<code>role</code>	Which logged-in user the browser starts as. Default from <code>app.config.ts</code> ; <code>test.use({ role: 'admin' })</code> to change, <code>null</code> for logged out.
<code>page</code>	The Playwright page, plus automatic capture of browser console errors and failed network calls for the bug report.
<code>api</code>	REST client (base URL and token from settings) for fast test-data setup and checks.
<code>cleanup</code>	Register removal of created data; runs after the test, even on failure.
<code>driver</code> , <code>platform</code>	Appium session for native app tests; skipped automatically if the app isn't configured.
<code>report</code>	Automatic: links the test case, sets severity, and on failure adds the cause and a bug report.
app pages / screens	<code>loginPage</code> , <code>inventoryPage</code> , ... from the app's <code>fixtures.ts</code> .

5. Walkthrough: automating a module with Claude

This is the day-to-day workflow. The example automates a "registration" module of an app called `shop`; replace the names with yours.



Step 0: check which app you are on

`APP` in the root `.env` must be the app you work on (here `APP=shop`). Claude also derives the app from the file path you give it.

Step 1: put the requirement in place

Copy `templates/requirement.md` to `apps/shop/requirements/web/SCRUM-123-registration.md` and fill it in (or paste the developer's notes / Jira story). The more exact it is (field rules, exact error texts, test IDs), the better the tests. The companion guide lists what a complete file contains and has a full example.

Step 2: generate test cases

```
/qa-testcases apps/shop/requirements/web/SCRUM-123-registration.md
```

Claude reads the requirement (and may open the page in the MCP browser to confirm fields) and writes `apps/shop/test-cases/web/registration.testcases.md` in the format of `templates/testcases.md`: IDs `TC-REG-01`, `TC-REG-02`, ..., type, priority, tags, steps, expected result and whether to automate. It covers every acceptance criterion, every required field and validation rule, boundaries, roles and UI states, and lists open questions. No code is written in this step. A baseline (stage `cases`) is recorded.

Step 3: review the test cases (the most important human step)

Read every row. Fix expected results, remove noise, add what's missing, answer the open questions with the developer/PO. A wrong expected result caught here never becomes a wrong test. Edit the MD file directly or ask Claude to change it.

Step 4: automate

```
/qa-automate apps/shop/test-cases/web/registration.testcases.md
```

Claude then:

1. opens the page in the MCP browser, logs in with the QA account of the right role, lists the real test IDs and triggers validation errors to read their exact texts;
2. writes the **model**, **page**, **fixture** registration, **data** and **spec**, reusing what already exists;
3. runs `npx tsc --noEmit` and the new spec, and fixes automation problems until the tests pass;
4. if the app behaves differently from the test case, keeps the assertion and reports it as a suspected bug;
5. runs coverage, records the baseline (stage `automated`) and reports back per test-case ID.

Step 5: review the generated code

Check it like a colleague's pull request: locators only in the model, one test per ID, expected texts match the test cases, no sleeps, no weakened checks, data cleaned up. Then commit.

Step 6: triage failures

```
/qa-fix apps/shop/tests/web/registration          # or a tag: /qa-fix @smoke   or empty: everything
```

Claude runs the tests, reads the failure details, reproduces each one in the browser and classifies it: **automation issue** (fixed in model/page/data), **framework issue** (fixed in `src/` only if generic), **application bug** (assertion kept, `knownBug(...)` added with the Jira key you give it) or **unclear requirement** (question to you).

Step 7: check coverage

```
npm run coverage:tc                                # every test-case ID: automated / MISSING / planned / manual, and
the % covered
```

Step 8: sign off

```
npm run req:baseline -- apps/shop/requirements/web/SCRUM-123-registration.md --stage signed-off --by
"Your Name"
```

Only QA records `signed-off`, after the results are accepted. Attach the Allure report to the Jira story.

5.1 When the developer changes the module later

Every requirement has a baseline: the version QA last processed. When the developer updates the MD file:

```
npm run req:status                                # all modules: stage, automation x/y, CHANGED flag
npm run req:diff -- apps/shop/requirements/web/SCRUM-123-registration.md
# ## Fields
#   ~ CHANGED Password: Error message: "Password is required" -> "Password must be at least 8
characters"
#   + ADDED    Remember me (Type: checkbox; ...)
/qa-update apps/shop/requirements/web/SCRUM-123-registration.md
```

`/qa-update` changes only what is affected: new cases get the next free IDs, changed cases keep their IDs, removed behaviour is retired with `Automate: no` (never deleted or renumbered). It updates the tests, runs them plus the module's smoke tests, and gives you a re-test list. Sign off again when done.

Don't edit by hand

Never edit files in `requirements/.baseline/` and never renumber test-case IDs: reports, coverage and history depend on them.

6. Working in sprints

A project is delivered in sprints, and each sprint has several user stories. Usually there is a web app, and sometimes a mobile app as well. This chapter shows how the per-module workflow from chapter 5 fits into that rhythm.

6.1 Who does what

Who	What they do with the framework
QA	Owns the framework and every file in it: requirement files, test cases, code and reports.
Developers	Don't use the framework. With every build they send one MD file per story in their own format , the build details and the credentials QA needs. The checklist to send them once is <code>templates/dev-handover.md</code> .

6.2 Starting a new web app

Do this once per project, before the first sprint. Chapter 7 explains each setting in more detail.

1. **Create the app folder:** `npm run new:app -- prj --platforms web,api` (leave out `api` if there are no API tests). It creates `apps/prj/` with `requirements/`, `test-cases/`, `sprints/`, the code folders, a starter `app.config.ts`, `fixtures.ts` and `.env`.
2. **Make it the active app:** `APP=prj` in the root `.env`.
3. **Fill in `app.config.ts`:** `baseUrl` (the QA URL), `testIdAttribute` (ask the developers: `data-testid`, `data-test` or `data-qa`), and `auth.roles` with a `defaultRole`.
4. **Fill in `apps/prj/.env`** with the accounts the developers shared (`USER_USERNAME`, `USER_PASSWORD`, ...). For email OTPs, the `MAIL_*` settings, then `npm run mail:check`.
5. **Automate the login first**, because every test starts logged in: requirement `requirements/web/PRJ-001-login.md` → `/qa-testcases` → review → `/qa-automate`. Ask Claude to wire the login steps into `auth.login`, then run `npm run auth`: every role must log in.
6. **Put the framework in Git** and commit.
7. **Send the developers** `templates/dev-handover.md`, so they know what to send with each build.

First sprint tips

- The login tests are the first `@smoke` tests. Tag the main happy path of each new story `@smoke` so the smoke suite grows every sprint.
- On machines with `NODE_ENV=production`, install with `npm install --include=dev`, or the Playwright packages are left out.
- While working on one spec: `npx playwright test <file> --project=web-chrome`.

6.3 Where the files go

<code>apps/<app>/</code>	
<code>requirements/</code>	ONE file per story per platform, written and owned by QA
<code> web/PRJ-101-login.md</code>	template format; developer notes are merged into it
<code> mobile/PRJ-140-login.md</code>	
<code> .baseline/</code>	versions QA has processed (written by the commands)
<code>test-cases/web/login.testcases.md</code>	QA's test cases (<code>/qa-testcases</code>)
<code>sprints/</code>	
<code> sprint-03.md</code>	sprint tracker (<code>templates/sprint.md</code>)
<code> sprint-03/from-dev/2026-10-15/</code>	developer MDs exactly as received, never edited

```
PRJ-101-login.md
PRJ-102-dashboard.md
.env                                credentials (never committed)
```

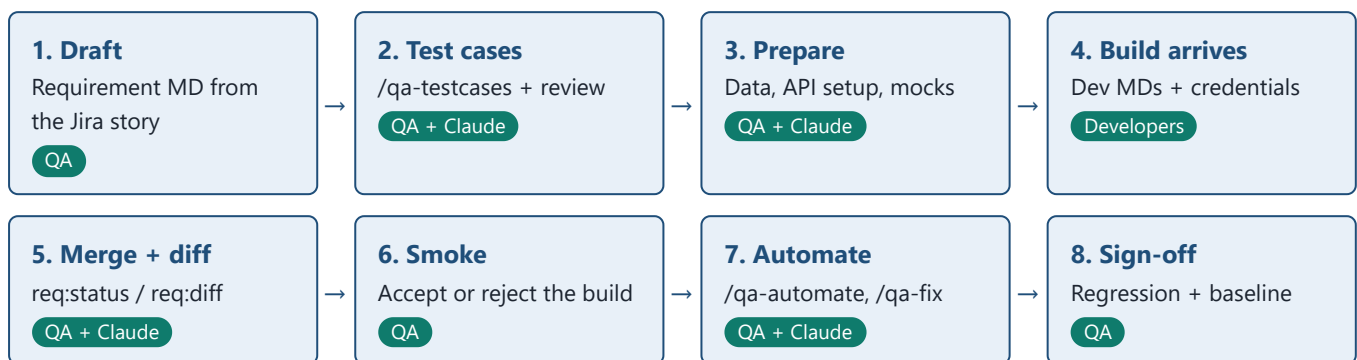
MD file	Written by	Where it goes	Edited later?
QA's MD, from the Jira story	QA	requirements/web/<JIRA>-<module>.md	Yes: developer notes are merged into it; the tests are built from it
Developers' MD, sent with a build	Developers	sprints/sprint-NN/from-dev/<build date>/<JIRA>-<module>.md	No: kept exactly as received
Test cases	QA + Claude	test-cases/web/<module>.testcases.md	Yes

Use the **same Jira key** in both file names (`PRJ-101-login.md` in `requirements/web/` and in `from-dev/2026-10-15/`), so each developer file clearly belongs to one QA file. A mobile version of the story goes in `requirements/mobile/` .

Don't sort requirements or test cases by sprint

Baselines are keyed by the file's path under `requirements/` . A story that changes again in sprint 5 keeps the same file, so `req:status` and `req:diff` keep working. Moving the file would lose its history. Which stories belong to which sprint is recorded only in `sprints/sprint-NN.md` .

6.4 The sprint, step by step



When	What QA does
Sprint planning	Copy <code>templates/sprint.md</code> to <code>sprints/sprint-NN.md</code> and list the stories. For each story, write <code>requirements/<web mobile>/<JIRA>-<module>.md</code> from the Jira story in the <code>templates/requirement.md</code> format. Test IDs are <code>TBD</code> , because nobody knows the locators yet.
Before the build	<code>/qa-testcases</code> on each requirement, then review it (chapter 5, steps 2–3). This records the cases baseline: QA's expectation . Prepare test data, API setup and <code>act.mockApi</code> states. Questions for the developers go in the sprint file.
Build day	Save, merge and compare (6.5). Run <code>npm run test:smoke</code> . If the build fails smoke, reject it and stop until a new build arrives.
Testing days	<code>/qa-automate</code> for each test-cases file. The MCP browser now finds the real locators in the build. <code>/qa-fix</code> for failures. Raise real bugs in Jira and add <code>knownBug(...)</code> to the test.
Bug-fix builds	New developer MDs go in a new date folder. Merge again, then <code>req:diff</code> → <code>/qa-update</code> .

When	What QA does
Sprint end	<code>npm run test:regression</code> (earlier sprints included), <code>npm run coverage:tc</code> , a clean <code>req:status</code> , and a signed-off baseline per story. Fill in the sign-off part of the sprint file.

6.5 Build day: save, merge, compare

1. **Save** each developer MD unchanged in `sprints/sprint-NN/from-dev/<YYYY-MM-DD>/` and add a row to the "Builds received" table of the sprint file. These copies are the record of what was delivered.
2. **Credentials:** put them in `apps/<app>/ .env` (6.6), then run `npm run auth`.
3. **Merge** each developer MD into its requirement file. Ask Claude:

```
Merge the developer notes in apps/prj/sprints/sprint-03/from-dev/2026-10-15/PRJ-101-login.md into
apps/prj/requirements/web/PRJ-101-login.md. Keep the template structure, fill in the build
details,
test IDs and exact messages they give, and add a Change log row "Merged dev notes, build 1.4.0".
List every point where the developer notes contradict what was there.
```

4. **Compare:**

```
npm run req:status                                # merged stories show CHANGED
npm run req:diff -- apps/prj/requirements/web/PRJ-101-login.md  # QA's expectation vs. the build
```

Each difference is either **accepted** (then run `/qa-update` on the requirement, which updates only the affected test cases and tests) or **questioned** (ask the developer, note it in the sprint file, and raise a bug if the build is wrong).

Why QA drafts the requirement before the build

The test cases are ready when the build arrives, so testing starts on day one. The diff also shows exactly where the developers built something different from the story, which would otherwise go unnoticed.

6.6 Credentials

- Developers share credentials through the password manager or another secure channel, **never inside the MD files**.
- QA stores them only in `apps/<app>/ .env` (not committed). Requirement files and test code refer to the variable name (`ADMIN_PASSWORD`), never the value.
- A new role: add it to `auth.roles` in `app.config.ts` and its variables to `.env`, then run `npm run auth`.
- OTP or email verification: use the test inbox (`MAIL_*`, `npm run mail:check`) or a fixed QA-only code in `OTP_CODE`.

6.7 Web and mobile

The same story on web and mobile gets two requirement files (`requirements/web/` and `requirements/mobile/`) and two test-cases files, because the screens, test IDs and builds differ. If the mobile app arrives in a later sprint, its file is added then and the web file is not touched.

7. Adding a new application

```
npm run new:app -- customer-portal --platforms web,mobile-web,android
```

This creates `apps/customer-portal/` with the standard folders, a starter `app.config.ts`, `fixtures.ts` and `.env.example`. Then:

1. **app.config.ts**: name, platforms, QA base URL, the test-ID attribute the developers use (`data-testid`, `data-test`, `data-qa`, ...), the login roles and the login steps.
2. **.env**: copy `.env.example`, fill in accounts per role (the names you used in `app.config.ts`), API URL/token, mobile builds.
3. **Root .env**: `APP=customer-portal`, or set it per command (section 10).
4. Build the login page model and page first (with Claude: give it the login requirement), because `auth.login` uses it.
5. Run `npm run auth` to check every role can log in, then continue with modules (section 5).

```
// apps/customer-portal/app.config.ts
export default defineApp({
  name: 'Customer Portal',
  platforms: ['web', 'mobile-web', 'android'],
  web: { baseUrl: 'https://qa.portal.example.com', testIdAttribute: 'data-testid' },
  auth: {
    defaultRole: 'customer',
    roles: {
      customer: { usernameEnv: 'CUSTOMER_USERNAME', passwordEnv: 'CUSTOMER_PASSWORD' },
      admin: { usernameEnv: 'ADMIN_USERNAME', passwordEnv: 'ADMIN_PASSWORD' },
    },
    login: async (page, credentials) => {
      const loginPage = new LoginPage(page);
      await loginPage.open();
      await loginPage.login(credentials);
      await new DashboardPage(page).expectLoaded(); // wait until really logged in
    },
  },
});
```

Apps without a login simply leave out `auth`.

8. Writing and editing tests by hand

Claude writes most of the code, but you will read and adjust it. This is the reference.

8.1 Field types

Type	Filled by <code>fillForm</code> / <code>setValue</code> with	Value in data
<code>text</code> , <code>password</code> , <code>email</code> , <code>number</code> , <code>textarea</code>	<code>fill</code> (passwords masked in the report)	string
dropdown	<code>select</code> : native <code><select></code> or custom (MUI, Ant, React-Select...): opened, option clicked	string or string[]
date	<code>setDate</code> : native date input needs <code>YYYY-MM-DD</code> ; custom pickers get the displayed format	string
checkbox	<code>check</code>	boolean
radio	<code>chooseRadio</code> : locator points at the group, option chosen by its label	string
file	<code>upload</code>	path or paths
<code>button</code> , <code>link</code> , <code>label</code>	not filled: clicked or read	(excluded)

8.2 Locators, in order of preference

Web locator	Example	Use when
<code>testId</code>	<code>{ testId: 'register-email-input' }</code>	Always, if the element has one
<code>role + name</code>	<code>{ role: 'button', name: 'Create account' }</code>	No test ID; accessible element
<code>label</code>	<code>{ label: 'Email address' }</code>	Form fields with a proper label
placeholder	<code>{ placeholder: 'Search products' }</code>	Inputs without a label
text	<code>{ text: 'No orders yet' }</code>	Static text
css	<code>{ css: '.toast-success' }</code>	Last resort
xpath	(avoid)	Only if nothing else works

Mobile: `accessibilityId` → `id` → `text` → platform-specific (`androidUiAutomator`, `iosPredicate`,
`iosClassChain`). A field can have one locator for both platforms or `{ android: {...}, ios: {...} }`.

8.3 Special cases

```
// Locator that depends on data (one button per product): a function in the MODEL returning a field
export const InventoryDynamicFields = {
  addToCartButton: (product: string): WebField => ({
    label: `Add to cart: ${product}`, type: 'button', locator: { testId: `add-to-
cart-${slug(product)}` },
  })),
};

// Custom dropdown whose options are not role="option"
country: { label: 'Country', type: 'dropdown', locator: { testId: 'country-select' },
```

```

    option: (v) => ({ css: `.menu-item:text-is("${v}")` }) },

// Field inside an iframe (payment widget, embedded editor)
cardNumber: { label: 'Card number', type: 'text', locator: { placeholder: 'Card number' }, frame:
'#payment-frame' },

```

8.4 Actions (`this.act`) and checks (`this.verify`) on web

Actions	Checks
<code>goto(path)</code> , <code>click</code> , <code>fill</code> , <code>clear</code> , <code>select</code> , <code>setDate</code> , <code>check</code> , <code>chooseRadio</code> , <code>upload</code> , <code>download</code> (returns file path), <code>press</code> , <code>hover</code> , <code>handleNextDialog(accept)</code> , <code>getText</code> , <code>getAllTexts</code> , <code>isVisible</code> , <code>setValue</code> , <code>fillForm(model, data)</code> , <code>mockApi(url, response)</code> , <code>locate(field)</code> (raw Playwright locator)	<code>visible</code> , <code>hidden</code> , <code>text</code> , <code>containsText</code> , <code>value</code> , <code>count</code> , <code>enabled(field, true false)</code> , <code>checked</code> , <code>url</code> , <code>title</code> . All auto-wait and retry.

Group several actions into one named report step with `step('Log in as "bob"', async () => {...})` from `@core/utils/step`.

8.5 Test data, roles, mocking

```

test.use({ role: 'admin' }); // whole file runs as admin

test('TC-ORD-07 | Admin can delete an order', { tag: ['@regression', '@TC-ORD-07'] },
  async ({ api, cleanup, ordersPage }) => {
    // create data through the API (fast), and always register its removal
    const { body } = await api.post<{ id: string }>('/api/v1/orders', { data: { item: 'Pen', qty: 1 }
});
    cleanup.add(`Delete order ${body.id}`, () => api.delete(`/api/v1/orders/${body.id}`, {
expectStatus: 204 }));
    ...
  });

test('TC-ORD-09 | Server error shows a banner', ..., async ({ ordersPage }) => {
  await ordersPage.act.mockApi(/\/api\/v1\/orders/, { status: 500, json: { error: 'boom' } }); //
before the request
  await ordersPage.open();
  ...
});

```

Unique values: `uniqueEmail()`, `uniqueId('ORD-')`, `randomDigits(10)` from `@core/utils/random`.

9. Native mobile tests (Appium)

Optional. Until a mobile app is configured, mobile tests are skipped automatically, so the rest works without this.

9.1 Setup

```
npm i -g appium
appium driver install uiautomator2      # Android
appium driver install xcuitest          # iOS (macOS + Xcode only)
appium                                  # start the server (default http://127.0.0.1:4723), keep it
running
```

Android needs Android Studio (SDK + an emulator) or a real device with USB debugging. In `apps/<app>/.env` set `ANDROID_APP` (path to the .apk) **or** `ANDROID_APP_PACKAGE` + `ANDROID_APP_ACTIVITY` for an installed app, and `ANDROID_DEVICE` (e.g. `emulator-5554`). iOS: `IOS_APP` or `IOS_BUNDLE_ID`, plus `IOS_DEVICE`. Mobile accounts go in `MOBILE_USERNAME` / `MOBILE_PASSWORD` (or as your app defines them).

9.2 Writing mobile tests

Same pattern as web: `defineMobileFields` models in `models/mobile`, screens extending `BaseScreen` in `screens/`, specs in `tests/mobile`. Actions: `tap`, `type`, `setSwitch`, `choose`, `getText`, `isDisplayed`, `swipe`, `hideKeyboard`, `back`, `setValue`, `fillForm`. Checks: `visible`, `hidden`, `text`, `containsText`. One Appium session per test; on failure a screenshot and the screen's XML source are attached.

Claude can't see native apps

The MCP browser only works for web. For native screens, put the accessibility IDs in the module MD file (from the developers or Appium Inspector). `/qa-automate` asks for them if they are missing.

10. Running tests locally and in CI

Command	What it runs
<code>npm test</code>	Everything for the active app (all its projects)
<code>npm run test:web</code>	Desktop Chrome
<code>npm run test:mobile-web</code>	Pixel 7 + iPhone 15 emulation
<code>npm run test:android / test:ios</code>	Native apps via Appium
<code>npm run test:smoke / test:regression</code>	By tag
<code>npx playwright test --grep @TC-LOGIN-02</code>	One test case
<code>npx playwright test <spec file> --project=web-chrome</code>	One spec on one project
<code>npm run test:headed / test:ui / test:debug</code>	Visible browser / interactive UI mode / step-through debugger
<code>npm run auth</code>	Refresh the saved logins (after password changes)
<code>npm run codegen</code>	Playwright recorder, handy to explore a page by hand

10.1 Choosing app and environment per run

Shell	Syntax
PowerShell	<code>\$env:APP='customer-portal'; \$env:TEST_ENV='staging'; npm test</code>
cmd.exe	<code>set APP=customer-portal && npm test</code>
Git Bash / macOS / Linux	<code>APP=customer-portal TEST_ENV=staging npm test</code>

Don't pass `--reporter` on the command line: it replaces the configured Allure reporter.

10.2 In CI

- Set `CI=true` (most CI systems do): 2 retries, 2 workers, and `test.only` left in code fails the run.
- Provide `APP`, `TEST_ENV` and every account/token as **CI secrets** (environment variables). They override the `.env` files, which are not in the repository.
- Steps: `npm ci` → `npx playwright install --with-deps chromium webkit` → `npx tsc --noEmit` → `npm test` → `npm run allure:generate` and publish `allure-report/`.
- `npm run coverage:tc -- <app> --strict` fails the build when a case marked `Automate: yes` has no test.
- Keep `allure-history.jsonl` between runs (cache/artifact) to get trend charts.

11. Reports and failure triage

11.1 The Allure report

`npm run report` builds and opens it. What you get with no extra code in the specs:

- A tree grouped **Epic** → **Feature** → **Story** → **test** (from `storyInfo`), with Jira links.
- For **every test**: its test case (preconditions, steps, expected result) from the `*.testcases.md` file, severity from its priority, platform and role.
- Readable steps (*Fill "Username" with "standard_user"*, passwords masked), screenshot, video and trace on failure.
- A **Dashboard** with status trends, severity and duration charts.

`npm run report:html` opens Playwright's own report, useful for the trace viewer (time-travel through each step).

11.2 Every failure comes with a cause and a bug report

The framework classifies each failure and the **Categories** tab sorts them:

Category	Typical causes	Who looks
Product bugs	Wrong text or value, wrong page/URL, element missing/disabled when it shouldn't be	Developers
Automation issues	Locator not found or not unique, setup/configuration, error in test code	QA
Environment problems	Server unreachable, slow response, timeouts	DevOps / re-run
Flaky	Passed only after a retry	QA

Each failed test has a **Bug report** attachment: steps to reproduce with the real data, expected vs actual, environment, browser or device, page URL, browser console errors, failed API calls, and a text version to paste into Jira. The cause is a strong hint, not a verdict: QA (or `/qa-fix`) confirms it.

11.3 Known bugs

```
test('TC-REG-05 | Duplicate email is rejected', { tag: ['@regression', '@TC-REG-05'] }, async ({
  registerPage }) => {
  knownBug('SCRUM-456', 'Duplicate email is accepted');    // first line of the test
  ...                                                       // assertions unchanged
});
```

The test stays green while the bug exists and is linked to it in the report. Once developers fix the bug, the test turns red ("passed unexpectedly"): that's the signal to remove the `knownBug` line.

11.4 Run history: every report is kept

The working folders (`allure-results/`, `allure-report/`, `test-results/`) are overwritten by each run. So that nothing is lost, every run also saves its own report permanently:

```
reports/
  index.html                                list of every saved run (npm run reports)
```

```
saucedemo/  
  2026-10-05_17-47-38_qa_PASSED/  
    report.html  
    summary.json  
  2026-10-06_09-12-03_staging_FAILED/  
    ...
```

full Allure report of that run: one file, opens offline date, app, environment, projects, counts, command

- `npm run reports` opens the list: date, app, environment, status, passed/failed/flaky/skipped, duration, the command that was run, and an **Open report** link. Use the filter box to find a run by date, app or status.
- Each `report.html` is self-contained (screenshots and failure details included). Copy it, attach it to Jira or email it.
- The run is saved automatically at the end of `npm test` and every `test:*` command. The login refresh alone (`npm run auth`) is not saved.
- Settings in the root `.env`: `REPORT_KEEP_DAYS=30` deletes runs older than 30 days (default `0` keeps everything); `REPORT_ARCHIVE=off` stops saving, for example while re-running one test many times during debugging.
- A report is about 4 MB, more when failures include videos. `reports/` is not committed to git; back it up or move it to a shared drive if the team needs it.

12. Rules of the framework

These are also in `CLAUDE.md`, so Claude follows them. Reviewers check them on every change.

Do

- Import `test / expect` from `@apps/<app>/fixtures`
- Keep locators in models only; pages use `act / verify`
- Derive data types from models:
`FormData<typeof XFields>`
- One test per test-case ID, title `<ID> | <Title>`, tags `@smoke / @regression + @<ID>`
- `storyInfo({...})` in `beforeEach`
- Create data via `api` and register `cleanup.add(...)`
- Read settings with `getEnv / requireEnv`
- Run `npx tsc --noEmit` after every change
- Put app-specific code in `apps/<app>/`

Don't

- Import from `@playwright/test` in specs
- Use `waitForTimeout` or any sleep
- Write raw locators in pages, screens or specs
- Weaken an assertion to match a bug: use `knownBug`
- Renumber test-case IDs or delete retired cases
- Edit `requirements/.baseline/` by hand
- Put app-specific code in `src/` (change `src/` only for generic improvements, and run every app's tests)
- Print, commit or email `.env` files or saved logs

13. Working well with Claude

- **Start with the slash commands.** They carry the whole procedure. Plain prompts are fine for questions and small changes.
- **Give file paths, not descriptions.** `/qa-automate apps/shop/test-cases/web/cart.testcases.md` beats "automate the cart".
- **Good input, good output.** Vague requirements produce vague tests. Fix the MD file rather than patching tests afterwards.
- **Review before you accept.** Read test cases and code diffs. Claude asks before running commands or editing files; read what it asks.
- **Be specific when correcting.** "TC-REG-04 expects the wrong text: the spec says 'Enter a valid email address'" is better than "fix the test".
- **One module per conversation.** Start a new conversation for a new module; long conversations get summarised and lose detail.
- **Tell it about bugs.** When you've raised a Jira bug, give Claude the key so it can add `knownBug`.
- **Teach the rules once.** If you keep giving the same correction, add it to `CLAUDE.md` (team-wide, via a PR) so every conversation follows it.
- **Secrets stay local.** Claude reads test accounts from `.env` to log in but is instructed never to print them. Never paste real production credentials into a conversation.

14. Troubleshooting

Symptom	Fix
<code>APP</code> is not set / Unknown APP "x"	Set <code>APP</code> in the root <code>.env</code> to a folder name under <code>apps/</code> . The message lists the valid names.
<code>npm install</code> says "up to date" but <code>node_modules</code> is empty, or <code>npx tsc</code> says "This is not the tsc command you are looking for"	Your machine has <code>NODE_ENV=production</code> , so npm skips dev dependencies (all of this project's packages are). Run <code>npm install --include=dev</code> , or remove that environment variable.
Missing setting X in <code>apps/<app>/.env</code>	Add the variable to the app's <code>.env</code> (see its <code>.env.example</code>).
The <code>setup</code> project fails ("log in as <role>")	Wrong account in the app <code>.env</code> , expired password, or the login flow changed: check <code>auth.login</code> in <code>app.config.ts</code> . Then <code>npm run auth</code> .
Tests suddenly start on the login page	Saved sessions expired: <code>npm run auth</code> .
All mobile tests are skipped	Expected until <code>ANDROID_APP</code> / <code>ANDROID_APP_PACKAGE</code> (or the iOS equivalent) is set and Appium is running.
<code>/mcp</code> doesn't show playwright, or it times out	Approve the server when asked; run the <code>claude mcp add</code> command from 3.4; reload VS Code; start a new conversation. First start downloads the package: retry once it has finished.
<code>/qa-...</code> commands missing	VS Code must be opened on the project root (the folder with <code>CLAUDE.md</code> and <code>.claude/</code>).
<code>APP=x npm test</code> doesn't work on Windows	That's Bash syntax; use the PowerShell or cmd form from 9.1.
Type errors after a change	<code>npx tsc --noEmit</code> shows the file and line. Usual cause: a field renamed in the model but not in data or page.
"locator resolved to N elements"	The test ID isn't unique: ask developers for unique IDs (lists: append the record ID) or narrow the locator in the model.
Report is empty or shows old results	Results are cleared at the start of each run; run the tests, then <code>npm run report</code> . Older runs: <code>npm run reports</code> (10.4).
A test passes locally but fails in CI	Check the CI secrets (accounts, <code>TEST_ENV</code>), then the trace in the report: often timing or data left by another test.

Appendix A. Command cheat sheet

In the Claude panel

Command	Input	Output
/qa-testcases	apps/<app>/requirements/<web mobile>/<JIRA>-<module>.md	test-cases/<module>.testcases.md + open questions
/qa-automate	apps/<app>/test-cases/.../<module>.testcases.md	Model, page/screen, data, spec; tests run until green
/qa-fix	Spec path, folder, @tag , or empty	Fixed automation issues; table of causes and actions
/qa-update	The changed requirement MD	Updated cases and tests for what changed; re-test list
/mcp		Status of the Playwright MCP server

In the terminal

Command	Purpose
npm install · npm run setup	Install dependencies · install browsers
npx tsc --noEmit	Type check (after every change)
npm test · test:web · test:mobile-web · test:android · test:ios	Run tests
npm run test:smoke · test:regression	Run by tag
npm run test:headed · test:ui · test:debug	Watch / interactive / debug
npm run auth	Refresh saved logins
npm run report · report:html	Allure report · Playwright report
npm run reports	List of every saved run, with links to each run's report
npm run coverage:tc [-- <app>] [--strict]	Which test-case IDs are automated
npm run req:status	Every module's stage and CHANGED flag
npm run req:diff -- <md>	What changed in a requirement since QA's baseline
npm run req:baseline -- <md> --stage <cases automated signed-off> [--by "name"]	Record the processed version (QA records signed-off)
npm run new:app -- <name> --platforms web,android	Create a new application folder
npm run docs:pdf	Rebuild the PDF guides from docs/*.html

Appendix B. Glossary

Term	Meaning
Agent	An AI that can take actions (edit files, run commands, use tools) in a loop to reach a goal, not just answer text.
Allure	The HTML test report used by this framework.
Appium	Server that automates native Android and iOS apps.
Baseline	The version of a requirement MD that QA last processed, stored in <code>requirements/.baseline/</code> , with a stage: <code>cases</code> , <code>automated</code> , <code>signed-off</code> .
CLAUDE.md	Project rules file Claude reads at the start of every conversation.
Fixture	Something Playwright prepares for a test and passes in by name (<code>page</code> , <code>api</code> , <code>loginPage</code> ...).
Locator	A description of how to find an element on a page or screen.
MCP	Model Context Protocol: standard way to give an AI assistant extra tools.
Model (field model)	File describing a screen's fields: label, type, locator, rules, messages.
Module MD / requirement	Markdown file describing one module or story: fields, rules, messages, test IDs. Input of the whole process.
Page / screen object	Class with the business actions of one web page or mobile screen.
Project (Playwright)	A named run configuration: browser/device + test folder (<code>web-chrome</code> , <code>android-app</code> ...).
Role	A type of user (customer, admin...) with its own test account and saved login session.
Slash command	Saved instructions for Claude, started with <code>/name</code> (our <code>/qa-*</code> commands).
Spec	A test file (<code>*.spec.ts</code>).
Test ID	Stable attribute developers add for automation (<code>data-testid</code> , accessibility ID).
Trace	Playwright's recording of a test (DOM snapshots, network, console) for time-travel debugging.
Web-first assertion	A check that retries until it passes or times out, instead of checking once.